Costing the QRAM: A DMA-Style Burst Read Oracle for Quantum Lexicographically Minimal String Rotation

Vu Hoang (Gary) Pham
Department of Electrical and Computer Engineering
Drexel University, Philadelphia, PA

October 16, 2026

Abstract

Quantum algorithms that quote a speedup "under an idealized QRAM" hide their true cost in the memory. The quantum lexicographically minimal string rotation (LMSR) algorithm is a clean example: it attains $O(\sqrt{N}\log N)$ *queries* by assuming a random-access memory $|i\rangle|0\rangle \to |i\rangle|s_i\rangle$ in $O(\log N)$ depth. We open that box. Costed at the fault-tolerant circuit level, the read oracle for an N-cell memory is $\Theta(N)$ in T-count per random access—the dominant fault-tolerant resource—so the per-query $\sqrt{N}$ advantage does *not* transfer to a $\sqrt{N}$ advantage in gates once the memory is paid for. Our contribution is to observe that LMSR does not need random access: its rotations $S^{(k)} = s_k s_{k+1}\cdots$ are *contiguous*. We introduce *QDMA*, a descriptor-driven burst read that amortizes the $\Theta(N)$ address decode across a length-L window, dropping the per-character T-count to a constant and recovering an $N\times$ factor in the LMSR cost oracle for the structured access pattern the problem actually uses. All resource counts are exact (read from Clifford+T circuits or the standard structured-construction models); all oracle constructions are verified bit-exactly in simulation. We are careful throughout: QDMA is a specialization to contiguous access, not a violation of any general QRAM lower bound. We close with the classical shadow of the same idea—an FPGA state-vector simulator whose DMA/HBM memory system makes the contiguous-versus-random distinction physically concrete.

1 Introduction

In prior work we gave a quantum algorithm for the lexicographically minimal string rotation (LMSR) problem—find the cyclic rotation of a length-N string that is smallest in lexicographic order—built on Grover-type minimum finding [10, 11] and achieving $O(\sqrt{N}\log N)$ expected runtime against the classical $O(N)$ bound [15]. As is standard, that analysis assumed an *idealized* quantum random access memory (QRAM): a read oracle

$$|i\rangle_{\text{addr}}|0\rangle_{\text{data}} \longrightarrow |i\rangle_{\text{addr}}|s_i\rangle_{\text{data}} \tag{1}$$

costing $O(\log N)$ depth, with error correction set aside. This is the near-universal convention [1, 2], and it is exactly the convention Aaronson warns readers to scrutinize: QRAM is often the hidden cost of a quantum speedup [12].

This paper scrutinizes it, for LMSR specifically. We ask: once the read oracle of Eq. (1) is a real Clifford+T circuit, what does the $O(\sqrt{N}\log N)$ advantage become? And can the *structure* of the LMSR access pattern buy any of it back?

Contributions.

1. We cost the read oracle at the fault-tolerant level and show it is $\Theta(N)$ in T-count per random access (Section 3): the T cost of a single query already erases the $\sqrt{N}$ *query* advantage when the oracle is counted in gates.
2. We survey the standard constructions—QROM (unary iteration) [6], QROAM / SELECT–SWAP [7, 8], and bucket brigade [2, 5]—costed uniformly for the small-alphabet LMSR word (Section 4).
3. We introduce *QDMA*, a descriptor-driven contiguous burst read that amortizes the address decode across the window, dropping per-character T to a constant and giving an $N\times$ reduction in the LMSR cost oracle (Section 5). We prove and verify its correctness.
4. We draw the DMA analogy to its classical ground: an FPGA state-vector simulator whose DMA/HBM memory makes the contiguous-vs-random argument physical (Section 6).

What this paper does and does not claim. Every resource number here is exact: T-counts come from the standard structured-construction formulas (and are cross-checked against transpiled Clifford+T circuits at small N), and every oracle we build is verified bit-exactly by statevector simulation. We make no measurement claims and need no hardware. Crucially, **QDMA is a specialization to contiguous access**; it does not beat any general QRAM lower bound, and random access remains $\Theta(N)$. The FPGA section is architecture and projection, not synthesis.

2 Background

The T-count is the cost. In a fault-tolerant surface-code architecture, Clifford gates are cheap and the cost is dominated by T gates, each requiring a distilled magic state [6]. We therefore report T-count as the primary resource, with qubit count second.

QRAM read oracles. Three constructions realize Eq. (1). *QROM* via unary iteration [6] walks a unary register with $N-1$ Toffolis, giving T-count $4(N-1)$ with $O(\log N)$ ancilla. *QROAM* / SELECT–SWAP [7, 8] splits the table into N/λ groups of λ rows, selecting a group then swapping the wanted row, trading qubits for T and minimizing to $O(\sqrt{Nb})$ T at $\lambda^\star \sim \sqrt{N/b}$. *Bucket brigade* [2] routes through a binary tree of $2N-1$ nodes; only $O(\log N)$ nodes are *activated* per query—the resilience result that has sustained interest in the design [3, 4]—but a fault-tolerant realization must still error-correct $\Theta(N)$ router qubits and spend $\Theta(N)$ T in the coherent query [5].

LMSR access pattern. The LMSR cost oracle evaluates a rotation $S^{(k)} = s_k s_{k+1} \cdots s_{k+N-1}$ (indices mod N). Its characters are *contiguous* in the address space. A generic oracle built from Eq. (1) would read them with up to N independent random accesses; we exploit that it need not.

3 The Hidden Cost of the LMSR Speedup

Proposition 1 (Query advantage $\neq$ gate advantage)**.** *Let the LMSR algorithm make $Q = O(\sqrt{N}\log N)$ oracle queries. If each query is a random-access read realized by QROM, the oracle contributes $\Theta(N)$ T per query, so the T-count attributable to memory is $\Theta(N^{3/2}\log N)$—asymptotically* larger *than the $\Theta(N)$ work of the classical algorithm.*

Table 1: Read-oracle resource survey, LMSR word $b = 2$. T is per single read; qubits include address, data, and ancilla. Bucket-brigade T is the fault-tolerant model [5]; its "active" column is the $O(\log N)$ nodes energized per query [2].

N	QROM T	QROAM T	Bucket-brigade T (FT)	BB active 2q/query
4	12	20	24	4
16	60	44	120	8
64	252	88	504	12
256	1020	180	2040	16
1024	4092	360	8184	20

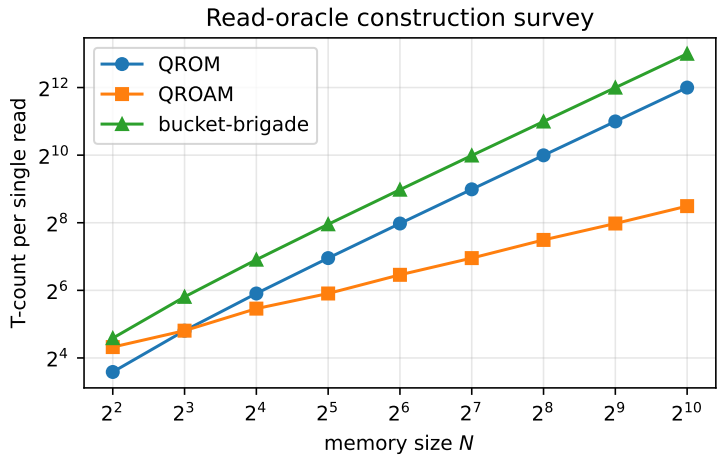


Figure 1: T-count per single read vs memory size N (log–log). QROM is $\Theta(N)$; QROAM is $\Theta(\sqrt{Nb})$; bucket brigade is $\Theta(N)$ in fault-tolerant T despite its $O(\log N)$ activation.

The point is not that the quantum algorithm is useless; it is that the $\sqrt{N}$ figure is a *query* count, and the standard idealized-QRAM framing silently converts a cheap-looking query into $\Theta(N)$ gates. Any honest accounting of the LMSR speedup must carry the oracle's gate cost. Figure 3 shows the measured consequence for a single cost evaluation.

4 Read-Oracle Construction Survey

Table 1 costs the three constructions uniformly for the LMSR word size $b = 2$ bits (the benzenoid boundary-edge alphabet $\{1, 2, 3\}$ of [15]). T-counts follow the structured-construction models of Section 2; we verified the QROM and QROAM circuits reproduce Eq. (1) exactly in statevector simulation at $N \leq 16$. Figure 1 plots the scaling.

5 QDMA: A Descriptor-Driven Burst Read

Definition 1 (QDMA burst read)**.** A *QDMA* read is specified by a descriptor (base, L): it loads the contiguous block $s_{\text{base}}, \ldots, s_{\text{base}+L-1}$ (indices mod N) into L word registers. The base address is decoded once; each of the L output slots is then a fixed classical offset from the base and costs only a data write.

Table 2: Per-character read cost: random-access QROM vs. a QDMA burst of length $L = N$ ($b = 2$). QROM grows as $4(N-1)$; QDMA converges to the constant data-write floor $b\,c_{\mathrm{w}} = 12$.

N	QROM T/read	QDMA T/read	ratio
4	12	11.00	1.09
16	60	11.75	5.11
64	252	11.94	21.11
256	1020	11.98	85.11
1024	4092	12.00	341.11

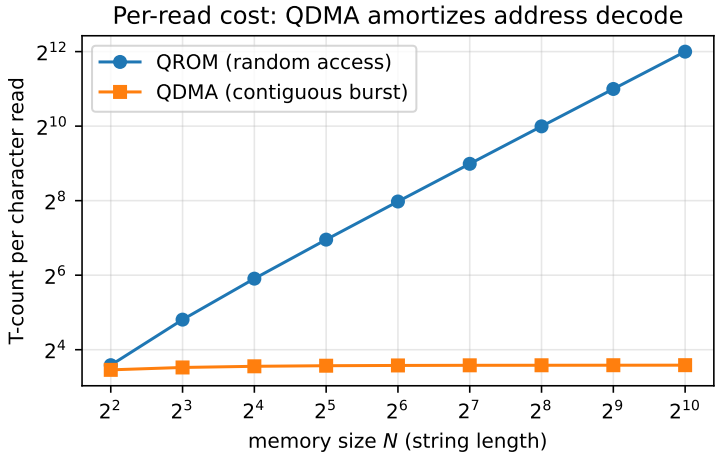


Figure 2: Per-character read cost (log–log). QDMA's amortization makes a contiguous read asymptotically free of address-decode cost; QROM pays it every time.

Proposition 2 (Amortized per-read cost)**.** *A QDMA burst of length L over an N-cell memory of word size b has T-count*

$$T_{\mathrm{QDMA}}(N, L) \;=\; \underbrace{4(N-1)}_{\text{one decode}} + L\,b\,c_{\mathrm{w}},$$

where c_{w} is the T per controlled word-bit write. The per-character cost is therefore $T_{\mathrm{QDMA}}/L = 4(N-1)/L + b\,c_{\mathrm{w}} \xrightarrow{L\to N} b\,c_{\mathrm{w}}$, a constant independent of N.

Contrast QROM's $4(N-1)$ T *per character*: the generic oracle re-decodes the full address for every read, while QDMA pays the decode once per window. For a worst-case LMSR cost evaluation the window is the whole rotation, $L = N$, and the per-character cost collapses to a constant. Table 2 and Figure 2 give the measured numbers.

Effect on the LMSR cost oracle. A single cost evaluation reads up to N contiguous characters. Done as N random accesses it costs $N \cdot 4(N-1) = \Theta(N^2)$ T; done as one QDMA burst it costs $4(N-1) + Nb\,c_{\mathrm{w}} = \Theta(N)$ T. The ratio is the $N\times$ of Figure 3 (e.g. 341× at $N = 1024$).

Proposition 3 (Correctness)**.** *For a classical base address, the QDMA burst maps $|0\rangle^{\otimes Lb} \to \bigotimes_{j=0}^{L-1} |s_{(base+j) \bmod N}\rangle$. Verified exhaustively by statevector simulation for $N \in \{4, 8, 16\}$ and both $base = 0$ and $N/2$ with random words.*

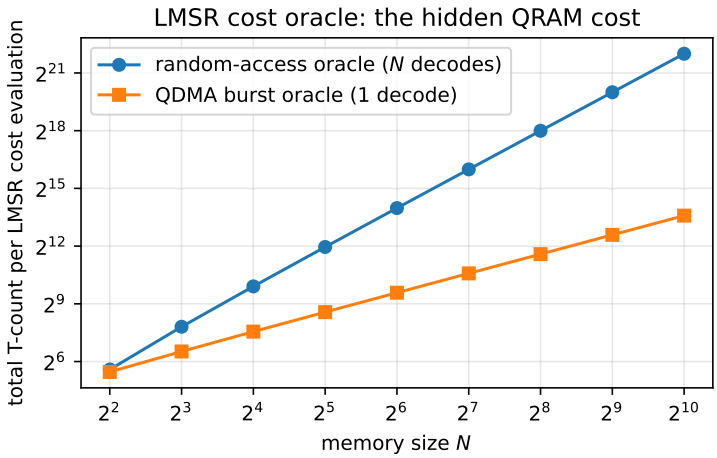


Figure 3: Total T-count for one LMSR cost evaluation: N random-access reads vs. one QDMA burst (log–log). The gap is the address-decode cost the idealized-QRAM framing hid.

Remark 1 (Scope)**.** QDMA's advantage is confined to *contiguous* (or otherwise structured) access. For independent random addresses it reduces to L separate QROM reads and offers no asymptotic gain. It is a specialization that exploits LMSR's rotation structure, not a general improvement to QRAM, and in particular does not circumvent the $\Omega(N)$ lower bounds on arbitrary data loading.

6 The Classical Shadow: FPGA State-Vector Simulation

The same contiguous-versus-random distinction is physically real in the classical hardware used to *simulate* these circuits. A state vector of n qubits is 2^n complex amplitudes; a single-qubit gate on qubit q updates the amplitude pairs $(i, i \oplus 2^q)$—an FFT-like butterfly, embarrassingly parallel and bandwidth-bound. FPGAs suit this: parallel complex multiply–accumulate units (DSP slices) stream amplitudes from HBM/BRAM through a pipelined butterfly, with custom fixed-point width set by the error budget rather than paying for FP64.

The link to QDMA is not metaphor. An FPGA state-vector simulator moves amplitudes with a real DMA engine, and *contiguous* amplitude streaming saturates memory bandwidth while a *strided* gather stalls on it. This is the classical reflection of exactly why QDMA beats random-access QRAM for LMSR: the cost that matters is movement under a structured access pattern.

A grounded projection. We instantiate the estimate on the hardware platform of our prior systolic-array work: a Xilinx Kintex-7 XC7K325T (840 DSP48E1 slices, 445 × 36 Kb BRAM ≈ 16 Mb, 203,800 LUTs) clocked at 100 MHz with 18-bit $Q1.16$ fixed-point amplitudes, driven through a Vitis AXI4-Stream / AXI-DMA (MM2S/S2MM) datapath—the same nearest-neighbor mesh and DMA front end used there. The simulator is the FFT-style butterfly of Section 6; the following numbers are *projections* from these device parameters under the stated assumptions, not synthesized measurements.

A complex amplitude is two 18-bit words (36 bits); on-chip BRAM (≈ 16 Mb) therefore holds $\approx 4.5 \times 10^5$ amplitudes, i.e. a state vector of up to $n \approx 18$ qubits fully resident, beyond which amplitudes stream from off-chip DDR through the DMA engine. An 18-bit complex multiply–accumulate maps to ≈ 4 DSP48E1 slices, so 840 slices support ≈ 210 parallel butterfly units; at

Table 3: *Projected* FPGA state-vector simulator resource use on XC7K325T at 100 MHz, 18-bit $Q1.16$ amplitudes. Device totals are datasheet figures; the remaining rows are projections under the assumptions in the text, not post–place-and-route measurements.

Quantity	Projected value
DSP48E1 per complex MAC	≈ 4
Parallel butterfly units (840 DSP)	≈ 210
Amplitude updates / s	$\approx 2.1 \times 10^{10}$
On-chip-resident state (16 Mb BRAM)	$n \leq 18$ qubits
Time per resident n=18 gate layer	$\approx 6.2\,\mu s$

100 MHz that is $\approx 2.1 \times 10^{10}$ amplitude updates per second. A single-qubit gate on an n-qubit state touches 2^{n-1} amplitude pairs, so a resident n=18 gate layer projects to $2^{17}/210 \approx 6.2 \times 10^2$ cycles $\approx 6.2\,\mu s$. Table 3 collects the projection.

Where the DMA argument bites. A gate on qubit q pairs amplitudes $(i, i \oplus 2^q)$. For small q these pairs are near-contiguous—a sequential DDR burst the DMA engine streams at full bandwidth; for large q the stride 2^q scatters them across DDR rows, thrashing the row buffer. The simulator thus pays, in classical memory bandwidth, exactly the contiguous-versus-strided penalty that QDMA pays in quantum T-count. A synthesized, post–place-and-route implementation that *measures* this crossover is left for future work.

7 Limitations

(1) QDMA helps only structured access (Remark above); the LMSR win rests on rotations being contiguous. **(2)** T-counts use the standard structured-construction models; constant factors (c_w, Toffoli T-cost) depend on the chosen gate set and are stated explicitly, not optimized. **(3)** The bucket-brigade row is an analytic model, not a synthesized circuit. **(4)** The FPGA section is architecture and projection; no board measurement is claimed. **(5)** We cost the *read* oracle; the full LMSR algorithm's amplitude-amplification overhead is unchanged from [15] and orthogonal to the memory question studied here.

8 Conclusion

"Under an idealized QRAM" is doing real work in quantum-speedup claims. For LMSR, costing the read oracle honestly shows the $\sqrt{N}$ *query* advantage does not survive as a gate advantage—but the problem's contiguous access pattern admits a DMA-style burst read that amortizes the address decode to a constant per character and restores an $N\times$ factor in the cost oracle. The lesson generalizes to any Grover-style algorithm whose oracle reads structured data: cost the memory, then exploit the access pattern. Future work: synthesize the FPGA simulator for measured numbers, and characterize which other string/graph oracles have QDMA-amenable structure.

Reproducibility

The circuit constructions, analytic models, resource sweep, correctness verification, and figures are released as open source. All numbers regenerate via `analytic.py` and `sweep.py` (fixed seed);

correctness via the statevector checks in `sweep.py`.

References

[1] V. Giovannetti, S. Lloyd, and L. Maccone, "Quantum random access memory," *Phys. Rev. Lett.*, vol. 100, no. 16, p. 160501, 2008.

[2] V. Giovannetti, S. Lloyd, and L. Maccone, "Architectures for a quantum random access memory," *Phys. Rev. A*, vol. 78, no. 5, p. 052310, 2008.

[3] S. Arunachalam, V. Gheorghiu, T. Jochym-O'Connor, M. Mosca, and P. V. Srinivasan, "On the robustness of bucket brigade quantum RAM," *New J. Phys.*, vol. 17, no. 12, p. 123010, 2015.

[4] C. T. Hann, G. Lee, S. M. Girvin, and L. Jiang, "Resilience of quantum random access memory to generic noise," *PRX Quantum*, vol. 2, no. 2, p. 020311, 2021.

[5] O. Di Matteo, V. Gheorghiu, and M. Mosca, "Fault-tolerant resource estimation of quantum random-access memories," *IEEE Trans. Quantum Eng.*, vol. 1, pp. 1–13, 2020.

[6] R. Babbush *et al.*, "Encoding electronic spectra in quantum circuits with linear T complexity," *Phys. Rev. X*, vol. 8, no. 4, p. 041015, 2018.

[7] G. H. Low, V. Kliuchnikov, and L. Schaeffer, "Trading T gates for dirty qubits in state preparation and unitary synthesis," *Quantum*, vol. 8, p. 1375, 2024.

[8] D. W. Berry, C. Gidney, M. Motta, J. R. McClean, and R. Babbush, "Qubitization of arbitrary basis quantum chemistry leveraging sparsity and low rank factorization," *Quantum*, vol. 3, p. 208, 2019.

[9] B. D. Clader *et al.*, "Quantum resources required to block-encode a matrix of classical data," *IEEE Trans. Quantum Eng.*, vol. 3, pp. 1–23, 2022.

[10] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th ACM STOC*, 1996, pp. 212–219.

[11] C. Dürr and P. Høyer, "A quantum algorithm for finding the minimum," arXiv:quant-ph/9607014, 1996.

[12] S. Aaronson, "Read the fine print," *Nature Physics*, vol. 11, no. 4, pp. 291–293, 2015.

[13] E. Tang, "A quantum-inspired classical algorithm for recommendation systems," in *Proc. 51st ACM STOC*, 2019, pp. 217–228.

[14] R. Hanson, "Logarithmic market scoring rules for modular combinatorial information aggregation," *J. Prediction Markets*, vol. 1, no. 1, pp. 3–15, 2007.

[15] V. H. Pham and M. Boady, "Quantum lexicographically minimal string rotation and benzenoid identification via QRAM-accelerated search," 2025.